YTC Alumni → ShiurPod Integration Plan

Self-contained briefing — sufficient to execute end-to-end with no prior conversation context.

How to use this document. If you are a Claude instance opening this PDF cold, you have everything needed to execute the integration. Section 1 is a 60-second orientation. Section 2 lists every decision that has already been made — do not relitigate these unless the user explicitly asks. Section 3 is the ShiurPod codebase reference (file paths, line numbers, key APIs you will touch). Section 4 is the YTC source reference. Sections 6–13 are phase-by-phase build instructions with code snippets you can lift directly. Section 14 is the verification checklist. Appendices contain verbatim copy targets, Firebase config, and out-of-scope follow-ups.

Table of contents

1. Brief for cold reader

What is being built

ShiurPod is a Torah audio podcast app (Expo SDK 54, React Native, expo-router, TypeScript, Drizzle ORM + Postgres backend). It already ships to Android and iOS with a working downloads / queue / mini-player / push-notifications stack. The user maintains it solo. Source: their local repo at `C:\Users\Guest2\Documents\Kosher-Feed`.

YTC Alumni is a separate Yeshiva Toras Chaim Alumni app (Expo SDK 52, also React Native + expo-router, with a Firebase JS SDK 11 backend on the Firebase project `toras-chaim-shiurim`). Source: `https://github.com/abbrach1/ytcalumni1` — the relevant variant is the `expo-app/` subfolder. The repo also contains parallel iOS-Swift, Android-Kotlin, and an older RN copy; ignore those for porting purposes.

Goal. Add a code-gated YTC section to ShiurPod (Android-only). User flow:

1. User enters an unlock code in ShiurPod Settings.
2. A YTC entry appears (Settings link; tab visibility is decided in §2).
3. Tapping it opens a Firebase email/password login screen (YTC's own Firebase project).
4. On successful sign-in, an approval check (Firestore lookup) decides whether the user sees the YTC tabs (Home / Shiurim / Events / Contacts) or a 'pending approval' screen.
5. All audio plays through ShiurPod's existing player. YTC notifications arrive via a backend bridge (see §13) and route into the YTC subtree on tap.

Constraints

- Android-only. No iOS work. This eliminates the APNs delegate conflict between expo-notifications and Firebase Messaging.
- No new native modules on the recommended path. The Firebase JS SDK is pure JS; no `@react-native-firebase/*` required for v1.
- Quarantine. All YTC code lives under `app/ytc/`, `lib/ytc/`, `contexts/YtcAuthContext.tsx`, `constants/ytcColors.ts`. Removable in a single PR.
- Lazy Firebase init. `initializeApp()` must not run at app cold-start. It runs only when a YTC route mounts (§9).
- RemoteConfig from day one. The unlock code lives in the existing `RemoteConfigContext` with a fallback constant. Rotation does not require a release.

2. Decisions already made (locked in)

These were debated and resolved in the planning conversation. Do not relitigate unless the user explicitly raises them. Each row gives the decision, the rationale, and where it surfaces in this plan.

#	Decision	Rationale	Surfaces in
D1	Code lives quarantined under ap[illegible]	[illegible]	[illegible] rs.ts.
D2	Unlock gate uses RemoteConfig ([illegible]	[illegible] ShiurPod [illegible]	[illegible] ready wires RemoteConfig.
D3	Unlock UI is a dedicated route (ap[illegible]	[illegible]	[illegible] ated route matches existing flows (feedbac
D4	Firebase init is lazy. lib/ytc/fireba[illegible]	[illegible]	[illegible] ing) for users who never unlock.
D5	Audio plays through ShiurPod's e[illegible]	[illegible]	[illegible] oid).
D6	YTC episodes use id prefix 'ytc:' s[illegible]	[illegible]	[illegible] plays.
D7	Playback positions are dual-writt[illegible]	[illegible]	[illegible]
D8	Notifications use a backend bridg[illegible]	[illegible]	[illegible]
D9	Notification payload contract reu[illegible]	[illegible]	[illegible] 'ytc-events', 'ytc-contacts'). Option
D10	A 'ytc' Android notification chann[illegible]	[illegible] mute YTC pushes.	§13
D11	Sign-out semantics: locking from [illegible]	[illegible]	[illegible]
D12	Audio refactor uses ShiurPod's pla[illegible]	[illegible] droid race for focus.	§12
D13	YTC plays do NOT count toward S[illegible]	[illegible] by 'ytc:' id pr[illegible]	[illegible] D, §12
D14	Downloads/offline for YTC shiuri[illegible]	[illegible] Appendix D as a follo[illegible]	[illegible]
D15	Firestore rules MUST be audited b[illegible]	[illegible] the security posture of a Fire[illegible]	[illegible] project the team may not own.
D16	iOS port is OUT OF SCOPE.	User specified Android-only. Re-enables APNs work later if iOS is added.	—

3. ShiurPod codebase reference

Verified against the working tree on 2026-05-07. Line numbers may drift; treat them as anchors, not exact addresses. Search by symbol where the reference matters.

3.1 Routes (app/)

- Tab routes (app/(tabs)/): index (home), following, favorites, downloads, settings. 5 tabs; downloads + settings hidden on web via `href: isWeb ? null : undefined`.
- Stack siblings (app/): onboarding, player (modal), queue (modal), all-shiurim, all-maggidei-shiur, category/[id], stats, storage, debug-logs, legal, messages, feedback, podcast/[id], maggid-shiur/[author], +native-intent, +not-found.
- All registered in `app/_layout.tsx` via `<Stack.Screen name=...>`.

3.2 Episode and Feed types

Path: `lib/types.ts`

```
export interface Feed {
  id: string;
  title: string;
  rssUrl: string;
  imageUrl: string | null;
  description: string | null;
  author: string | null;
  categoryId: string | null;
  categoryIds?: string[];
  isActive: boolean;
  isFeatured: boolean;
  scheduledPublishAt: string | null;
  lastFetchedAt: string | null;
  createdAt: string;
  sourceNetwork: string | null;
  tatSpeakerId?: number | null;
}

export interface Episode {
  id: string;
  feedId: string;
  title: string;
  description: string | null;
  audioUrl: string;
  duration: string | null;          // ISO 8601 duration or "HH:MM:SS"
  publishedAt: string | null;       // ISO date string
  guid: string;
  imageUrl: string | null;
  adminNotes: string | null;
  sourceSheetUrl: string | null;
  createdAt: string;
  tatLectureId?: number | null;
  noDownload?: boolean;
}
```

3.3 AudioPlayerContext

Path: `contexts/AudioPlayerContext.tsx` (~1380 lines).
Hook: `useAudioPlayer()`.
Critical signature: `playEpisode(episode: Episode, feed: Feed): Promise<void>` — requires real `Episode` and `Feed` objects (not just a URL). This is what drives the synthetic-wrapper requirement in §12.

Full context value shape (relevant subset):

```
interface AudioPlayerContextValue {
  currentEpisode: Episode | null;
  currentFeed: Feed | null;
  playback: {
    isPlaying: boolean;
    isLoading: boolean;
    positionMs: number;
    durationMs: number;
    playbackRate: number;
    playbackError?: string | null;
  };
  playEpisode: (episode: Episode, feed: Feed) => Promise<void>;
  pause: () => Promise<void>;
  resume: () => Promise<void>;
  seekTo: (positionMs: number) => Promise<void>;
  skip: (seconds: number) => Promise<void>;
  setRate: (rate: number) => Promise<void>;
  stop: () => Promise<void>;
  getSavedPosition: (episodeId: string) => Promise<number>;
  // ...queue, sleepTimer, recently-played, etc.
}
```

Position persistence: `savePosition(episodeId, feedId, positionMs, durationMs)` writes to AsyncStorage AND POSTs to `/api/playback-positions`. The server endpoint at `server/routes.ts:764` does not validate that episodeId/feedId reference real rows — synthesized YTC ids (`ytc:<shiurId>`) go through cleanly.

3.4 Settings screen pattern

Path: `app/(tabs)/settings.tsx` (~853 lines)

Each section is a `<View style={[styles.section, ...]}>` wrapper containing an ALL-CAPS section header (`styles.sectionHeader`) and a `<View style={styles.sectionContent}>` wrapper around `<SettingRow>` rows. Rows use `<View style={styles.divider} />` as separators. Pickers go through `<OptionPickerModal>`. There is no inline TextInput / Alert.prompt / <Modal> pattern in this file — drives D3.

SettingRow signature (top of the same file, ~line 30):

```
interface SettingRowProps {
  icon: React.ReactNode;
  label: string;
  value?: string;
  subtitle?: string;
  onPress?: () => void;
  rightElement?: React.ReactNode;
  autoFocus?: boolean;
}
```

3.5 Push notifications

Path: `lib/push-notifications.ts`

Existing channels (`setupPushNotificationChannels`, line ~104):

- default — MAX importance, vibrate, light #1a73e8, sound default, badge
- new-episodes — same shape, used for new-episode pushes

Tap-data shape (`getNotificationData`, line ~335):

```
export function getNotificationData(response: Notifications.NotificationResponse): {
  episodeId?: string;
  feedId?: string;
  type?: string;
  screen?: string;        // <-- existing field, reused for YTC routing (D9)
  conversationId?: string;
} { /* ... */ }
```

Tap routing (`handleNotificationResponse` in app/_layout.tsx, line ~89):

```
if (data.screen === "messages") {
  router.push("/messages" as any);
} else if (data.feedId) {
  router.push(`/podcast/${data.feedId}` as any);
}
// YTC branch goes here (see §13).
```

3.6 RemoteConfigContext

Path: `contexts/RemoteConfigContext.tsx`

Fetches from `{api}/api/config` on mount; merges with `DEFAULT_CONFIG`; caches in AsyncStorage under `@shiurpod_remote_config`. The `RemoteConfig` interface uses `[key: string]: any` so adding new keys requires NO type extension. Hook: `useRemoteConfig().config.ytcUnlockCode`.

To add the unlock code key, ALSO update DEFAULT_CONFIG with a fallback string:

```
const DEFAULT_CONFIG: RemoteConfig = {
  // ...existing keys...
  ytcUnlockCode: "1234",  // fallback; real value served from /api/config
};
```

3.7 Other relevant surfaces

- `contexts/AudioPlayerContext.tsx`'s `recentlyPlayed` state is in-memory; no DB write needed for YTC recently-played to participate.
- `components/MiniPlayerHost.tsx` renders the global mini-player; YTC reuses it without changes.
- `lib/error-logger.ts` exports `addLog(level, msg, stack?, channel?)`; use channel `'ytc'` for all YTC-related logs.
- `lib/query-client.ts` exports `apiRequest`, `queryClient`, `getApiUrl`.
- Path alias `@/` resolves to repo root (per tsconfig.json).

4. YTC source reference

4.1 Where to fetch

Repo: `https://github.com/abbrach1/ytcalumni1`. The variant we port is `expo-app/`. Other folders (`android-app/`, `ytcalumni1/` [iOS Swift], `react-native/`) are reference only — do not port from them.

Clone command:

```
git clone --depth 1 https://github.com/abbrach1/ytcalumni1 /tmp/ytc-source
```

4.2 expo-app file inventory

Path	Lines	Role / port action
expo-app/services/firebase.ts	~180	Copy verbatim into lib/ytc/firebase.ts; wrap in lazy init (§9).
expo-app/contexts/AuthContext.tsx	~70	Copy verbatim into contexts/YtcAuthContext.tsx; mount in app/ytc/_layout.tsx or
expo-app/contexts/AudioContext.tsx	~200	DROP. Replaced by ShiurPod's AudioPlayerContext via adapter (§12).
expo-app/components/MiniPlayer.tsx	~80	DROP. ShiurPod's MiniPlayerHost handles this.
expo-app/constants/Colors.ts	~30	Copy into constants/ytcColors.ts (rename to avoid collision with ShiurPod's Colo
expo-app/app/_layout.tsx	~40	Logic merged into app/ytc/_layout.tsx (§10). Do not copy as-is.
expo-app/app/(auth)/_layout.tsx	5	Copy into app/ytc/(auth)/_layout.tsx.
expo-app/app/(auth)/login.tsx	259	Copy into app/ytc/(auth)/login.tsx; apply refactor recipe (§11).
expo-app/app/(auth)/pending.tsx	117	Copy into app/ytc/(auth)/pending.tsx; apply refactor recipe.
expo-app/app/(tabs)/_layout.tsx	82	Copy into app/ytc/(tabs)/_layout.tsx; remove the in-file MiniPlayer wrapper.
expo-app/app/(tabs)/index.tsx	365	Copy; apply refactor recipe + audio adapter.
expo-app/app/(tabs)/shiurim.tsx	514	Copy; apply refactor recipe + audio adapter.
expo-app/app/(tabs)/events.tsx	208	Copy; apply refactor recipe.
expo-app/app/(tabs)/contacts.tsx	362	Copy; apply refactor recipe.
expo-app/types/*	—	Copy types into types/ytc.ts (Shiur, Event, Rebbe, Announcement, etc.).
expo-app/components/*	—	Inspect and port any non-MiniPlayer components alongside the screens that use

4.3 Firestore collections used (by expo-app)

Collection	Purpose
shiurim	Audio shiurim catalog: title, rebbe, date, audioUrl, pdfUrl, tags, playCount, downloadCount, series, descrip
events	Events / simchos calendar: eventName, personFamily, type, date, location, time, imageUrl, description
announcements	Banner announcements (filter where enabled==true): title, content, type, date, enabled
carouselImages	Home carousel: url, caption, order
rebbeim	Rebbe directory: name, title, email, phone, photoUrl
alumniContactSubmissions	Alumni-submitted contacts; status='approved' shown in directory
alumniDatabase	Alumni records keyed by lowercase email — primary approval source
approvedEmails	Fallback approval list (by doc id OR by 'email' field)
admins	Admin email list, keyed by lowercase email
accessRequests	Pending requests from non-approved sign-ups (write-only from app)
users/{uid}/preferences/	Per-user data (positions, saved shiurim) — used by native YTC apps; mirror writes here (D7)
settings/featuredShiur	Configurable featured shiur; not used by expo-app port directly

4.4 Approval logic (verbatim, from ytc/services/firebase.ts)

```
export async function checkUserApproval(email: string): Promise<{ approved: boolean; admin: boolean }> {
  const normalizedEmail = email.toLowerCase();
  let approved = false;
  let admin = false;
  try {
    // 1. Check alumniDatabase by lowercase email as doc id
    const alumniDoc = await getDoc(doc(db, "alumniDatabase", normalizedEmail));
    if (alumniDoc.exists()) approved = true;
    // 2. Fallback: approvedEmails by doc id
    if (!approved) {
      const approvedDoc = await getDoc(doc(db, "approvedEmails", normalizedEmail));
      if (approvedDoc.exists()) approved = true;
    }
    // 3. Fallback: approvedEmails by 'email' field
    if (!approved) {
      const q = query(collection(db, "approvedEmails"), where("email", "==", normalizedEmail));
      const snap = await getDocs(q);
      if (!snap.empty) approved = true;
    }
    // 4. Admin check
    const adminDoc = await getDoc(doc(db, "admins", normalizedEmail));
    if (adminDoc.exists()) admin = true;
  } catch (e) {
    console.warn("Approval check error:", e);
  }
  return { approved, admin };
}
```

5. Architecture & file layout

```
NEW files:
  app/
    ytc-unlock.tsx unlock code entry screen (D3)
    ytc/
      _layout.tsx auth-gate layout, mounts YtcAuthProvider
      (auth)/
        _layout.tsx
        login.tsx
        pending.tsx
      (tabs)/
        _layout.tsx
        index.tsx
        shiurim.tsx
        events.tsx
        contacts.tsx
  contexts/
    YtcAuthContext.tsx Firebase auth + approval state
  lib/
    ytc/
      firebase.ts lazy init + Firestore queries
      unlock.ts AsyncStorage flag + RemoteConfig check
      audio-adapter.ts synthetic Episode/Feed wrappers (§12)
      positions.ts dual-write to ShiurPod + Firestore (§12)
  constants/
    ytcColors.ts navy/gold/cream palette
  types/
    ytc.ts Shiur, Event, Rebbe, Announcement

MODIFIED files:
  package.json add 'firebase' dep
  metro.config.js only if firebase resolution complains
  contexts/RemoteConfigContext.tsx add ytcUnlockCode to DEFAULT_CONFIG
  app/_layout.tsx extend handleNotificationResponse + add /ytc + /ytc-unlock to Stack
  app/(tabs)/settings.tsx add YTC Alumni section
  lib/push-notifications.ts add 'ytc' Android channel
  contexts/AudioPlayerContext.tsx (optional) skip /api/playback-positions for ytc:* ids; alternate path is
in lib/ytc/positions.ts
```

6. Phase 0 — Pre-flight checks

Run these before writing any code. They take 5 minutes and prevent surprises.

1. Confirm working tree is clean: `git status`.
2. Confirm shiurpod's package.json does NOT already contain `firebase` or `@react-native-firebase/*` (greenfield install expected).
3. Confirm `contexts/RemoteConfigContext.tsx` exists and exports `useRemoteConfig`.
4. Confirm `contexts/AudioPlayerContext.tsx` exports `useAudioPlayer` with `playEpisode(episode, feed)`.
5. Confirm `lib/push-notifications.ts` exports `setupPushNotificationChannels` and `getNotificationData`.
6. Confirm `app/_layout.tsx` contains `handleNotificationResponse`.
7. Verify the YTC repo is reachable (`git ls-remote https://github.com/abbrach1/ytcalumni1`).
8. Open Appendix C and decide: who owns the firestore.rules audit? Block ship until done.

7. Phase 1 — Dependencies & Metro config

Time: ~30 min. Adds the Firebase JS SDK; tweaks Metro only if needed.

7.1 Add Firebase to package.json

```
npm install firebase@^11.1.0 --save
```

7.2 Verify Metro can resolve Firebase

Run `npx expo start --clear`. If you see errors about `.cjs` files or 'package.json exports', append to `metro.config.js`:

```
// metro.config.js (append before module.exports)
config.resolver.sourceExts.push("cjs");
config.resolver.unstable_enablePackageExports = false;
```

7.3 No app.json plugin changes required.

The existing `expo-notifications` plugin handles push for both apps. Do NOT add `googleServicesFile`; the Firebase JS SDK does not use it.

8. Phase 2 — Unlock gate

Time: ~1 hr. Adds the unlock flag, a route to enter the code, and a settings entry.

8.1 Add ytcUnlockCode to RemoteConfig

Edit `contexts/RemoteConfigContext.tsx` — add a key to `DEFAULT_CONFIG` (search for the literal block):

```
const DEFAULT_CONFIG: RemoteConfig = {
  // ...existing keys...
  recommendationsLimit: 10,
  ytcUnlockCode: "1234",  // FALLBACK ONLY; real value comes from /api/config
};
```

Companion server change: see Appendix E.

8.2 lib/ytc/unlock.ts

```
import AsyncStorage from "@react-native-async-storage/async-storage";

const UNLOCK_KEY = "@shiurpod_ytc_unlocked";

// In-memory listeners so the settings UI / tab bar can react without reload.
const listeners = new Set<() => void>();
function emit() { listeners.forEach(fn => fn()); }
export function onUnlockChanged(cb: () => void) {
  listeners.add(cb);
  return () => { listeners.delete(cb); };
}

export async function isUnlocked(): Promise<boolean> {
  try {
    return (await AsyncStorage.getItem(UNLOCK_KEY)) === "1";
  } catch {
    return false;
  }
}

/**
 * Validate the entered code against RemoteConfig.ytcUnlockCode and persist.
 * Caller passes in the current code from useRemoteConfig().
 */
export async function tryUnlock(entered: string, expected: string): Promise<boolean> {
  if (!expected || entered.trim() !== expected.trim()) return false;
  await AsyncStorage.setItem(UNLOCK_KEY, "1");
  emit();
  return true;
}

/** Lock + sign out of Firebase (D11). */
export async function lock(): Promise<void> {
  await AsyncStorage.removeItem(UNLOCK_KEY);
  // Lazy import so we never pull Firebase into the root bundle.
  try {
    const { firebaseSignOutIfInitialized } = await import("@/lib/ytc/firebase");
    await firebaseSignOutIfInitialized();
  } catch {}
  emit();
}
```

8.3 Hook for components

```
// lib/ytc/unlock.ts (continued)
import { useEffect, useState } from "react";

export function useYtcUnlocked(): boolean {
  const [unlocked, setUnlocked] = useState(false);
  useEffect(() => {
    let mounted = true;
    isUnlocked().then(v => mounted && setUnlocked(v));
    const off = onUnlockChanged(() => isUnlocked().then(v => mounted && setUnlocked(v)));
    return () => { mounted = false; off(); };
  }, []);
  return unlocked;
}
```

8.4 app/ytc-unlock.tsx (new screen)

Mirrors the visual style of feedback.tsx / messages.tsx (full-screen modal-style form).

```
import React, { useState } from "react";
import { View, Text, TextInput, Pressable, StyleSheet, Alert, KeyboardAvoidingView, Platform } from "react-native";
import { router, Stack } from "expo-router";
import { useSafeAreaInsets } from "react-native-safe-area-context";
import { Ionicons } from "@expo/vector-icons";
import { useAppColorScheme } from "@/lib/useAppColorScheme";
import Colors from "@/constants/colors";
import { useRemoteConfig } from "@/contexts/RemoteConfigContext";
import { tryUnlock } from "@/lib/ytc/unlock";
import { mediumHaptic } from "@/lib/haptics";

export default function YtcUnlockScreen() {
  const insets = useSafeAreaInsets();
  const isDark = useAppColorScheme() === "dark";
  const colors = isDark ? Colors.dark : Colors.light;
  const { config } = useRemoteConfig();
  const [code, setCode] = useState("");
  const [submitting, setSubmitting] = useState(false);

  const onSubmit = async () => {
    if (submitting) return;
    setSubmitting(true);
    const expected = (config as any).ytcUnlockCode || "";
    const ok = await tryUnlock(code, expected);
    setSubmitting(false);
    if (ok) {
      mediumHaptic();
      router.replace("/ytc");
    } else {
      Alert.alert("Incorrect code", "The access code you entered is not valid.");
    }
  };

  return (
    <KeyboardAvoidingView behavior={Platform.OS === "ios" ? "padding" : undefined} style={{ flex: 1, backgroundColor: c
      <Stack.Screen options={{ headerShown: false }} />
      <View style={{ paddingTop: insets.top + 12, paddingHorizontal: 20, flex: 1 }}>
        <Pressable onPress={() => router.back()} style={{ alignSelf: "flex-start", padding: 8 }}>
          <Ionicons name="close" size={24} color={colors.text} />
        </Pressable>
```

```
        <Text style={{ fontSize: 22, fontWeight: "700", color: colors.text, marginTop: 12 }}>YTC Alumni Access</Text>
        <Text style={{ fontSize: 14, color: colors.textSecondary, marginTop: 6 }}>
          Enter your access code to enable the YTC Alumni section.
        </Text>
        <TextInput
          value={code}
          onChangeText={setCode}
          placeholder="Access code"
          placeholderTextColor={colors.textSecondary}
          autoFocus
          autoCapitalize="none"
          autoCorrect={false}
          onSubmitEditing={onSubmit}
          style={{
            marginTop: 24, fontSize: 18, color: colors.text,
            borderWidth: 1, borderColor: colors.border, borderRadius: 10,
            paddingHorizontal: 14, paddingVertical: 12, backgroundColor: colors.surface,
          }}
        />
        <Pressable
          onPress={onSubmit}
          disabled={submitting || !code}
          style={{ marginTop: 16, backgroundColor: colors.accent, padding: 14, borderRadius: 10, opacity: submitting ||
          <Text style={{ color: "#fff", fontWeight: "600", textAlign: "center", fontSize: 16 }}>Unlock</Text>
        </Pressable>
      </View>
    </KeyboardAvoidingView>
  );
}
```

8.5 Settings section (modify app/(tabs)/settings.tsx)

Add a section near the bottom (above the existing 'About' / 'Legal' section). Insert after the existing sections close. Use the existing SettingRow + section patterns:

```
import { useYtcUnlocked, lock as lockYtc } from "@/lib/ytc/unlock";
// inside SettingsScreenInner:
const ytcUnlocked = useYtcUnlocked();

// JSX (place where appropriate):
<View style={[styles.section, { borderColor: colors.cardBorder }]}>
  <Text style={[styles.sectionHeader, { color: colors.textSecondary }]}>YTC ALUMNI</Text>
  <View style={[styles.sectionContent, { backgroundColor: colors.surface, borderColor: colors.cardBorder }]}>
    {ytcUnlocked ? (
      <>
        <SettingRow
          icon={<Ionicons name="school" size={20} color={colors.accent} />}
          label="YTC Alumni"
          subtitle="Enabled"
          onPress={() => { lightHaptic(); router.push("/ytc"); }}
        />
        <View style={[styles.divider, { backgroundColor: colors.border }]} />
        <SettingRow
          icon={<Ionicons name="lock-closed" size={20} color={colors.accent} />}
          label="Disable YTC access"
          onPress={() => {
            Alert.alert("Disable YTC", "This will sign you out of YTC and hide the section.", [
              { text: "Cancel", style: "cancel" },
              { text: "Disable", style: "destructive", onPress: () => { lockYtc(); } },
            ]);
```

```
          }}
        />
      </>
    ) : (
      <SettingRow
        icon={<Ionicons name="key" size={20} color={colors.accent} />}
        label="Enter access code"
        onPress={() => { lightHaptic(); router.push("/ytc-unlock"); }}
      />
    )}
  </View>
</View>
```

8.6 Stack registration (modify app/_layout.tsx)

Add two routes inside `RootLayoutNav`'s <Stack>:

```
<Stack.Screen name="ytc-unlock" options={{ headerShown: false, presentation: "modal", animation: "slide_from_bottom" }}
<Stack.Screen name="ytc" options={{ headerShown: false }} />
```

9. Phase 3 — Lazy Firebase service

Time: ~1 hr. Critical: initializeApp must NOT run at app cold-start.

9.1 lib/ytc/firebase.ts (lazy-init pattern)

```
/**
 * YTC Firebase service. ALL access goes through getYtcFirebase(); never import
 * 'firebase/app' directly elsewhere. The lazy init is what keeps Firebase out
 * of the root bundle for users who never unlock the section.
 */
import type { FirebaseApp } from "firebase/app";
import type { Auth, User } from "firebase/auth";
import type { Firestore, DocumentSnapshot } from "firebase/firestore";

// Public client config (safe to commit; identical to the value in
// ytcalumni1/expo-app/services/firebase.ts).
const firebaseConfig = {
  apiKey: "AIzaSyB-j6Itt_DKVLOm5BGsuygVUD6YoPKQyS8",
  authDomain: "toras-chaim-shiurim.firebaseapp.com",
  projectId: "toras-chaim-shiurim",
  storageBucket: "toras-chaim-shiurim.firebasestorage.app",
  messagingSenderId: "95643621522",
  appId: "1:95643621522:ios:a75e5f1bdfaba692986e4b",
};

let _initialized: { app: FirebaseApp; auth: Auth; db: Firestore } | null = null;
let _initPromise: Promise<typeof _initialized> | null = null;

export async function getYtcFirebase() {
  if (_initialized) return _initialized;
  if (!_initPromise) {
    _initPromise = (async () => {
      const { initializeApp, getApps } = await import("firebase/app");
      const { getAuth } = await import("firebase/auth");
      const { getFirestore } = await import("firebase/firestore");
      const app = getApps().length === 0 ? initializeApp(firebaseConfig) : getApps()[0];
      _initialized = { app, auth: getAuth(app), db: getFirestore(app) };
      return _initialized;
    })();
  }
  return _initPromise;
}

/** Sign out only if Firebase has been initialized. Safe to call from lock(). */
export async function firebaseSignOutIfInitialized() {
  if (!_initialized) return;
  const { signOut } = await import("firebase/auth");
  try { await signOut(_initialized.auth); } catch {}
}

// Optional: re-export onAuthStateChanged through the lazy path.
export async function subscribeAuth(cb: (user: User | null) => void): Promise<() => void> {
  const { auth } = await getYtcFirebase();
  const { onAuthStateChanged } = await import("firebase/auth");
  return onAuthStateChanged(auth, cb);
}
```

9.2 Firestore query helpers

Port the rest of `ytcalumni1/expo-app/services/firebase.ts` functions verbatim, but replace the static `db` reference with a call to `getYtcFirebase()`. Pattern:

```
// Original:  const snap = await getDocs(query(collection(db, "shiurim"), orderBy("date", "desc")));
// Lazy:
export async function fetchShiurim() {
  const { db } = await getYtcFirebase();
  const { collection, query, orderBy, getDocs } = await import("firebase/firestore");
  const snap = await getDocs(query(collection(db, "shiurim"), orderBy("date", "desc")));
  return snap.docs.map(docToShiur).filter(Boolean);
}
```

Apply this pattern to: `fetchShiurim`, `fetchMostRecentShiur`, `incrementPlayCount`, `fetchEvents`, `fetchUpcomingEvents`, `fetchAnnouncements`, `fetchCarouselImages`, `fetchRebbeim`, `fetchApprovedAlumni`, `checkUserApproval`, `submitAccessRequest`.

9.3 Bundle audit

After implementation, verify lazy init by:

1. Cold-starting the app without YTC unlocked.
2. Searching the Metro bundle for `'firebase/app'` or `'AIzaSyB-j6Itt_DKVLOm5BGsuygVUD6YoPKQyS8'`: it should appear in async chunks but not the root bundle.
3. Checking with the React Native debugger that `firebase/app`'s `initializeApp` is NOT called until the user navigates to /ytc-unlock or /ytc.

10. Phase 4 — YtcAuthContext + auth-gate layout

Time: ~1.5 hr. Provides Firebase auth state + approval flag, scoped to the /ytc subtree.

10.1 contexts/YtcAuthContext.tsx

```
import React, { createContext, useContext, useEffect, useState } from "react";
import type { User } from "firebase/auth";
import { getYtcFirebase, subscribeAuth } from "@/lib/ytc/firebase";
import { checkUserApproval } from "@/lib/ytc/firebase"; // export this function from firebase.ts

interface YtcAuthState {
  user: User | null;
  isApproved: boolean;
  isAdmin: boolean;
  isLoading: boolean;
}

interface YtcAuthContextValue extends YtcAuthState {
  signOut: () => Promise<void>;
  refreshStatus: () => Promise<void>;
}

const YtcAuthContext = createContext<YtcAuthContextValue>({
  user: null, isApproved: false, isAdmin: false, isLoading: true,
  signOut: async () => {}, refreshStatus: async () => {},
});

export function YtcAuthProvider({ children }: { children: React.ReactNode }) {
  const [state, setState] = useState<YtcAuthState>({
    user: null, isApproved: false, isAdmin: false, isLoading: true,
  });

  const updateApproval = async (user: User) => {
    if (!user.email) return;
    const { approved, admin } = await checkUserApproval(user.email);
    setState(prev => ({ ...prev, user, isApproved: approved, isAdmin: admin, isLoading: false }));
  };

  useEffect(() => {
    let unsub: (() => void) | null = null;
    (async () => {
      unsub = await subscribeAuth(async (user) => {
        if (user) await updateApproval(user);
        else setState({ user: null, isApproved: false, isAdmin: false, isLoading: false });
      });
    })();
    return () => { unsub?.(); };
  }, []);

  const signOut = async () => {
    const { firebaseSignOutIfInitialized } = await import("@/lib/ytc/firebase");
    await firebaseSignOutIfInitialized();
  };
  const refreshStatus = async () => { if (state.user) await updateApproval(state.user); };

  return (
    <YtcAuthContext.Provider value={{ ...state, signOut, refreshStatus }}>
      {children}
    </YtcAuthContext.Provider>
  );
}

export const useYtcAuth = () => useContext(YtcAuthContext);
```

10.2 app/ytc/_layout.tsx

```
import { useEffect } from "react";
import { Stack, router } from "expo-router";
import { YtcAuthProvider, useYtcAuth } from "@/contexts/YtcAuthContext";

function YtcGate() {
  const { user, isApproved, isLoading } = useYtcAuth();
  useEffect(() => {
    if (isLoading) return;
    if (!user) router.replace("/ytc/(auth)/login");
    else if (!isApproved) router.replace("/ytc/(auth)/pending");
    else router.replace("/ytc/(tabs)");
  }, [user, isApproved, isLoading]);
  return (
    <Stack screenOptions={{ headerShown: false }}>
      <Stack.Screen name="(auth)" />
      <Stack.Screen name="(tabs)" />
    </Stack>
  );
}

export default function YtcRootLayout() {
  return (
    <YtcAuthProvider>
      <YtcGate />
    </YtcAuthProvider>
  );
}
```

11. Phase 5 — Route porting (refactor recipe)

Time: ~6–8 hr. Mostly mechanical for 3 of 5 screens; the home and shiurim screens take audio integration work too (§12).

11.1 Refactor recipe (apply per-file)

For each ported file from `ytcalumni1/expo-app/app/`:

1. Replace `'../contexts/AuthContext'` → `'@/contexts/YtcAuthContext'`.
2. Replace `'../services/firebase'` → `'@/lib/ytc/firebase'`.
3. Replace `'../constants/Colors'` → `'@/constants/ytcColors'`.
4. Replace any `'../types'` → `'@/types/ytc'`.
5. Replace any `'../components/MiniPlayer'` imports — DROP the import; ShiurPod's MiniPlayerHost handles it.
6. Replace audio: `useAudio()` → `useAudioPlayer()` + the audio adapter (§12).
7. If a file imports `expo-av` directly, remove that import; route through ShiurPod's player.
8. Add the `'use client'`-equivalent or any TS strictness fixups your tsconfig requires.

11.2 Per-file notes

Target file	Notes beyond recipe
app/ytc/(auth)/_layout.tsx	Just a Stack with two screens. No special handling.
app/ytc/(auth)/login.tsx	Imports signInWithEmailAndPassword from firebas
app/ytc/(auth)/pending.tsx	Posts to accessRequests collection. Replace with the new lazy fetchers from lib/ytc/firebase.
app/ytc/(tabs)/_layout.tsx	DROP the wrapped MiniPlayer view at the bottom. ShiurPod's MiniPlayerHost</fo
app/ytc/(tabs)/index.tsx	Home screen. Fetches carousel, recent shiur, upcoming events, announcements. Replace play-shiur buttor
app/ytc/(tabs)/shiurim.tsx	Largest screen. Filtering + sorting in-memory. Each shiur tile triggers playYtcShiur(
app/ytc/(tabs)/events.tsx	Read-only list. Recipe alone is sufficient.
app/ytc/(tabs)/contacts.tsx	Read-only list. Phone/email links use Linking.openURL (already in expo). R

11.3 Provider order in app/ytc/_layout.tsx

YtcAuthProvider mounts inside the YTC subtree only. Do NOT add it to the root layout.

12. Phase 6 — Audio adapter

Time: ~2–3 hr. The most code-heavy phase.

12.1 lib/ytc/audio-adapter.ts

Synthesizes shiurpod-shaped Episode and Feed objects from a YTC Shiur, then plays via `useAudioPlayer().playEpisode`.

```
import type { Episode, Feed } from "@/lib/types";
import type { Shiur } from "@/types/ytc";

const YTC_FEED_PREFIX = "ytc:feed:";
const YTC_EPISODE_PREFIX = "ytc:";

/** Synthesize a virtual feed per rebbe. Used as Feed for ShiurPod's player. */
export function ytcRebbeToFeed(rebbeName: string): Feed {
  const id = `${YTC_FEED_PREFIX}${rebbeName.toLowerCase().replace(/\s+/g, "-")}`;
  return {
    id,
    title: rebbeName,
    rssUrl: "",                  // not a real RSS feed
    imageUrl: null,
    description: null,
    author: rebbeName,
    categoryId: null,
    isActive: true,
    isFeatured: false,
    scheduledPublishAt: null,
    lastFetchedAt: null,
    createdAt: new Date().toISOString(),
    sourceNetwork: "ytc",
  };
}

export function ytcShiurToEpisode(shiur: Shiur, feed: Feed): Episode {
  return {
    id: `${YTC_EPISODE_PREFIX}${shiur.id}`,
    feedId: feed.id,
    title: shiur.title,
    description: shiur.description ?? null,
    audioUrl: shiur.audioUrl,
    duration: null,              // ytc Shiur has no duration field; player computes from media
    publishedAt: shiur.date || null,
    guid: `${YTC_EPISODE_PREFIX}${shiur.id}`,
    imageUrl: null,
    adminNotes: null,
    sourceSheetUrl: shiur.pdfUrl ?? null,
    createdAt: shiur.date || new Date().toISOString(),
    noDownload: true,            // D14: downloads OOS for v1
  };
}

export function isYtcEpisodeId(id: string): boolean {
  return id.startsWith(YTC_EPISODE_PREFIX);
}
```

12.2 Play helper that wraps useAudioPlayer

```
// app/ytc/(tabs)/shiurim.tsx and index.tsx
import { useAudioPlayer } from "@/contexts/AudioPlayerContext";
import { ytcRebbeToFeed, ytcShiurToEpisode } from "@/lib/ytc/audio-adapter";
import { incrementPlayCount } from "@/lib/ytc/firebase";

function useYtcPlay() {
  const { playEpisode } = useAudioPlayer();
  return async (shiur: Shiur) => {
    const feed = ytcRebbeToFeed(shiur.rebbe || "YTC");
    const episode = ytcShiurToEpisode(shiur, feed);
    await playEpisode(episode, feed);
    incrementPlayCount(shiur.id).catch(() => {}); // Firestore, fire-and-forget
  };
}
```

12.3 Position dual-write (D7)

ShiurPod's AudioPlayerContext already POSTs to `/api/playback-positions` on every position change. To also mirror to Firestore `users/{uid}/preferences/positions/{shiurId}`, subscribe to position changes from inside the YTC subtree (so it only runs when YTC is active):

```
// lib/ytc/positions.ts
import { onPositionsChanged, loadPositions } from "@/contexts/AudioPlayerContext";
import { getYtcFirebase } from "@/lib/ytc/firebase";
import { isYtcEpisodeId } from "@/lib/ytc/audio-adapter";

let installed = false;
export function installYtcPositionMirror() {
  if (installed) return;
  installed = true;
  let writing: Promise<any> = Promise.resolve();
  onPositionsChanged(async () => {
    writing = writing.then(async () => {
      const positions = await loadPositions();
      const { db, auth } = await getYtcFirebase();
      const uid = auth.currentUser?.uid;
      if (!uid) return;
      const { doc, setDoc } = await import("firebase/firestore");
      for (const [id, p] of Object.entries(positions)) {
        if (!isYtcEpisodeId(id)) continue;
        const shiurId = id.slice("ytc:".length);
        await setDoc(doc(db, `users/${uid}/preferences/positions/${shiurId}`), {
          positionMs: p.positionMs,
          durationMs: p.durationMs,
          updatedAt: p.updatedAt,
        }, { merge: true });
      }
    }).catch(() => {});
  });
}
```

Call `installYtcPositionMirror()` once from inside `YtcAuthProvider` after the user is signed in. It is a no-op if YTC is not active.

12.4 Optional: skip ShiurPod position-sync for ytc:* ids

If polluting `playback_positions` on the server with synthesized YTC ids is undesirable, edit `contexts/AudioPlayerContext.tsx` in `syncPositionToServer` (line ~179) to early-return when `episodeId.startsWith('ytc:')`. Keep AsyncStorage write (needed for in-app resume). This is a one-line change:

```
async function syncPositionToServer(episodeId: string, feedId: string, positionMs: number, durationMs: number) {
  if (episodeId.startsWith("ytc:")) return;   // YTC mirrors via lib/ytc/positions.ts instead
  // ...existing implementation...
}
```

13. Phase 7 — Notifications (recommended path: backend bridge)

Time: ~3 hr (+server work). The recommended path keeps shiurpod's existing Expo Push pipeline as the only on-device receiver.

13.1 Architecture

YTC's notification sender currently emits to FCM topics (`all_users`, `announcements`, `new_shiurim`, `events`). We do NOT subscribe shiurpod to those topics directly (would require @react-native-firebase/messaging — Appendix F). Instead a server-side bridge does the topic-to-user fan-out and re-emits via Expo Push to shiurpod tokens linked to YTC users.

Diagram (text):

```
YTC server publishes to FCM topic 'new_shiurim'
        |
        v
[Bridge worker] subscribes to FCM topics OR receives webhook
        |
        v
For each YTC user mapped to a shiurpod expoPushToken,
emit Expo Push payload:
  {
    to: expoPushToken,
    title: "New shiur from Rabbi X",
    body: shiur.title,
    data: { screen: "ytc-shiurim", ytcShiurId: shiur.id, app: "ytc" },
    channelId: "ytc",
  }
```

13.2 Add 'ytc' Android channel

Edit `lib/push-notifications.ts` in `setupPushNotificationChannels`:

```
await Notifications.setNotificationChannelAsync("ytc", {
  name: "YTC Alumni",
  importance: Notifications.AndroidImportance.HIGH,
  vibrationPattern: [0, 250, 250, 250],
  lightColor: "#19263F",
  sound: "default",
  enableVibrate: true,
  showBadge: true,
});
```

13.3 Extend tap routing (app/_layout.tsx)

Inside `handleNotificationResponse`, before the existing branches:

```
import { isUnlocked } from "@/lib/ytc/unlock";
// ...inside handleNotificationResponse, in the setTimeout:
if (data.screen?.startsWith("ytc-")) {
  isUnlocked().then(ok => {
    if (!ok) {
      router.push("/(tabs)/settings" as any);
      return;
    }
    if (data.screen === "ytc-shiurim") router.push("/ytc/(tabs)/shiurim" as any);
    else if (data.screen === "ytc-events") router.push("/ytc/(tabs)/events" as any);
    else if (data.screen === "ytc-contacts") router.push("/ytc/(tabs)/contacts" as any);
    else router.push("/ytc/(tabs)" as any);
```

```
    // Optional deep link to a specific shiur if data.ytcShiurId exists.
  });
  return;
}
```

13.4 Extend getNotificationData (lib/push-notifications.ts)

```
export function getNotificationData(response: Notifications.NotificationResponse): {
  episodeId?: string;
  feedId?: string;
  type?: string;
  screen?: string;
  conversationId?: string;
  ytcShiurId?: string;   // <-- NEW
} {
  const data = response.notification.request.content.data as Record<string, any> | undefined;
  if (!data) return {};
  return {
    episodeId: data.episodeId, feedId: data.feedId, type: data.type,
    screen: data.screen, conversationId: data.conversationId,
    ytcShiurId: data.ytcShiurId,
  };
}
```

13.5 Device linking endpoint

On YTC successful login, the app posts to a new shiurpod endpoint:

```
POST /api/ytc/link-device
{ "deviceId": "<shiurpod-device-id>", "ytcUid": "<firebase-uid>" }
```

Server stores the mapping in a new table (Appendix E). The bridge worker uses it to translate YTC topic events into per-user Expo Push sends.

13.6 Client side of the link

```
// In contexts/YtcAuthContext.tsx, after isApproved becomes true:
useEffect(() => {
  if (!state.user || !state.isApproved) return;
  (async () => {
    try {
      const deviceId = await getDeviceId();
      await apiRequest("POST", "/api/ytc/link-device", {
        deviceId, ytcUid: state.user.uid,
      });
    } catch (e) {
      addLog("warn", `ytc link-device failed: ${(e as any)?.message || e}`, undefined, "ytc");
    }
  })();
}, [state.user, state.isApproved]);
```

14. Phase 8 — Verification checklist

Walk through every item before declaring done.

- Cold start (locked). Fresh install, no unlock. Search Metro bundle: 'firebase/app' should appear in async chunks only. `initializeApp` never called per debug breakpoint.
- Wrong code. /ytc-unlock with wrong code -> alert, no flag flip, no Firebase init.
- Right code. /ytc-unlock with correct code -> router.replace('/ytc') -> auth gate -> login. Settings UI updates without app reload.
- Sign up flow. Non-approved email -> /ytc/(auth)/pending. Tapping 'request access' creates accessRequests doc.
- Approved sign-in. Approved email -> /ytc/(tabs). Home shows carousel/recent/announcements without errors.
- Audio: YTC plays in shiurpod player. Tap a shiur -> mini-player appears -> full player opens. Episode title is correct, position bar moves.
- Audio: cross-app session. Start a regular shiurpod episode, then play a YTC shiur, then return to shiurpod. No double-audio, no orphaned focus.
- Audio: position resume. Play a YTC shiur 30s in, force-quit, reopen, return to /ytc -> player resumes at 30s (in-app) AND Firestore positions doc has matching value.
- Stats untouched. Listening stats screen does not show YTC titles. (D13)
- Notifications channel. adb settings UI shows 3 channels: default, new-episodes, ytc.
- Notification tap (locked). Send a test push with data.screen='ytc-shiurim'. Tap -> routes to /(tabs)/settings (because YTC is locked).
- Notification tap (unlocked). Same push, after unlock -> routes to /ytc/(tabs)/shiurim.
- Lock from settings. Disable YTC -> tab gone, /ytc-unlock fresh ask, Firebase signed out. /ytc routes return to login if visited directly.
- Sign-out independence. Sign out of shiurpod main account (if applicable) -> YTC unlock and YTC Firebase session both untouched.
- RemoteConfig rotation. Change ytcUnlockCode server-side -> next /ytc-unlock attempt with old code fails; new code works.
- No iOS regression. If iOS build is still produced (even though YTC is Android-only), confirm app boots normally and the YTC settings entry is hidden or non-functional. (Plan: still hide on iOS via Platform.OS check or just leave settings entry enabled but route to a 'Android only' notice.)
- Bundle size. Compare release APK size pre/post — Firebase JS adds ~200KB compressed. Acceptable.
- Firestore rules audit. Appendix C completed and signed off.

15. Effort estimate

Phase	Time	Risk areas
1. Deps & Metro	~30 min	Metro cjs / package-exports tweak
2. Unlock gate	~1 hr	—
3. Lazy Firebase service	~1 hr	Bundle audit takes longer if first-time
4. YtcAuthContext + gate	~1.5 hr	Lazy import boundaries
5. Route porting	~6–8 hr	Mechanical refactor; verify visual parity
6. Audio adapter	~2–3 hr	Position dual-write race conditions; play-count fire-and-forget timing
7. Notifications + bridge wire	~3 hr (+server)	Bridge worker is server-side work; channel + routing on-device is small
8. Verification	~1.5 hr	Audio session regression scenarios
Firestore rules audit	~1 hr	External dependency; do early
Total (client side)	~2.5–3 days	

Appendix A — Verbatim copy targets

Run from a freshly cloned `/tmp/ytc-source`. The 'shiurpod path' column is where the file lands in the shiurpod repo.

YTC source path	ShiurPod path
expo-app/services/firebase.ts	lib/ytc/firebase.ts (apply lazy-init wrapper §9)
expo-app/contexts/AuthContext.tsx	contexts/YtcAuthContext.tsx
expo-app/constants/Colors.ts	constants/ytcColors.ts
expo-app/types/index.ts (or types/*.ts)	types/ytc.ts
expo-app/app/(auth)/_layout.tsx	app/ytc/(auth)/_layout.tsx
expo-app/app/(auth)/login.tsx	app/ytc/(auth)/login.tsx
expo-app/app/(auth)/pending.tsx	app/ytc/(auth)/pending.tsx
expo-app/app/(tabs)/_layout.tsx	app/ytc/(tabs)/_layout.tsx
expo-app/app/(tabs)/index.tsx	app/ytc/(tabs)/index.tsx
expo-app/app/(tabs)/shiurim.tsx	app/ytc/(tabs)/shiurim.tsx
expo-app/app/(tabs)/events.tsx	app/ytc/(tabs)/events.tsx
expo-app/app/(tabs)/contacts.tsx	app/ytc/(tabs)/contacts.tsx

Appendix B — Firebase config & Firestore collections

Public client config (commit-safe, identical across all YTC variants):

```
{
  apiKey: "AIzaSyB-j6Itt_DKVLOm5BGsuygVUD6YoPKQyS8",
  authDomain: "toras-chaim-shiurim.firebaseapp.com",
  projectId: "toras-chaim-shiurim",
  storageBucket: "toras-chaim-shiurim.firebasestorage.app",
  messagingSenderId: "95643621522",
  appId: "1:95643621522:ios:a75e5f1bdfaba692986e4b"
}
```

Firestore collections used by ports here: see §4.3. FCM topics used by native YTC apps: `all_users`, `announcements`, `new_shiurim`, `events`.

Appendix C — Firestore rules audit checklist

Before ship, somebody who can read the YTC Firebase project needs to verify:

- Reads on collections `shiurim, events, announcements, carouselImages, rebbeim` are public OR scoped to authenticated users (and shiurpod users will be authenticated post-login).
- Reads on `alumniDatabase, approvedEmails, admins` are restricted such that an attacker with the apiKey + a Firebase test account cannot dump the email lists.
- Writes on `accessRequests, alumniContactSubmissions, simchaSubmissions` are rate-limited or otherwise abuse-resistant.

- Writes on `users/{uid}/preferences/*` are scoped to the owning uid (so dual-writes from shiurpod cannot tamper with other users' data).
- `shiurim.playCount` increment is allowed for any authenticated user (used by `incrementPlayCount`).

Appendix D — Out of scope / follow-ups

- Downloads / offline for YTC shiurim. Synthesized Episode has `noDownload: true` for v1. Adding offline support means deciding whether YTC files live in shiurpod's downloads dir alongside other shiurim, what storage budget gets allocated, and whether unsubscribing from YTC purges them.
- YTC plays counted in shiurpod stats. Currently filtered out (D13). If the user changes their mind, remove the filter — synthesized ids already make it easy to opt YTC in.
- iOS port. Not blocked architecturally — the JS SDK is cross-platform. APNs delegate concern returns; see Appendix F if path B is later chosen.
- Path B (RNFirebase Messaging). See Appendix F. Allows direct topic subscription without a backend bridge; tradeoff is native module + manifest-merge work.
- Auth biometrics on top of unlock code. Trivial to add via `expo-local-authentication` but currently not in scope.
- Code-rotation UX. If RemoteConfig rotates the unlock code, currently-unlocked devices stay unlocked. If the user wants forced re-validation, add a config version key and re-check.
- Featured shiur on home. Native YTC apps render `settings/featuredShiur`; the expo-app does not. If desired, add it during the home-screen port.

Appendix E — Server-side companion changes

Two changes to the shiurpod backend (server/):

1. /api/config: serve ytcUnlockCode

```
// server/routes.ts (or wherever /api/config is handled)
res.json({
  // ...existing fields...
  ytcUnlockCode: process.env.YTC_UNLOCK_CODE || "1234",
});
```

2. /api/ytc/link-device: store device <-> ytcUid mapping

```
// drizzle schema (shared/schema.ts or wherever table defs live)
export const ytcDeviceLinks = pgTable("ytc_device_links", {
  deviceId: text("device_id").primaryKey(),
  ytcUid: text("ytc_uid").notNull(),
  linkedAt: timestamp("linked_at").defaultNow().notNull(),
});

// server/routes.ts
app.post("/api/ytc/link-device", async (req, res) => {
  const { deviceId, ytcUid } = req.body;
  if (!deviceId || !ytcUid) return res.status(400).json({ error: "deviceId and ytcUid required" });
  await db.insert(ytcDeviceLinks)
    .values({ deviceId, ytcUid })
    .onConflictDoUpdate({ target: ytcDeviceLinks.deviceId, set: { ytcUid, linkedAt: new Date() } });
  res.json({ ok: true });
});
```

3. (Optional, for path A bridge) Worker that consumes YTC FCM topic events and sends Expo Push:

Implementation depends on where YTC's notification sender lives. Two patterns:

- Webhook — modify YTC's sender to POST topic events to `/api/ytc/topic-event` on shiurpod. Shiurpod looks up linked device tokens for that topic-equivalent audience and sends Expo Push.
- FCM consumer — a Node worker uses the Firebase Admin SDK to subscribe via the FCM HTTP v1 API (or a Firebase Cloud Function on the YTC project that fires on topic events) and POSTs to shiurpod.

Either way, the Expo Push payload should be:

```
{
  "to": "<expoPushToken>",
  "title": "<from topic event>",
  "body":  "<from topic event>",
  "data":  { "screen": "ytc-shiurim" | "ytc-events" | "ytc-home" | "ytc-contacts",
             "ytcShiurId": "<optional>", "app": "ytc" },
  "channelId": "ytc",
  "android": { "priority": "high" }
}
```

Appendix F — Notification path B (RNFirebase Messaging)

Reference only. Use this if D8 is reversed.

F.1 Dependencies

```
npm install @react-native-firebase/app @react-native-firebase/messaging --save
```

Add config plugin entries to app.json:

```
"plugins": [
  ...,
  "@react-native-firebase/app",
  "@react-native-firebase/messaging"
]
```

F.2 Place google-services.json

Download from the Firebase console for the `toras-chaim-shiurim` project (Android app section) and reference it from app.json:

```
"android": {
  "package": "com.shiurpod.app",   // or whatever shiurpod's existing package is
  "googleServicesFile": "./google-services.json"
}
```

F.3 Manifest-merge collision

Both `expo-notifications` and `@react-native-firebase/messaging` register a `FirebaseMessagingService` with intent filter `com.google.firebase.MESSAGING_EVENT`. Only one wins. Resolution pattern: let RNFirebase be the FCM receiver, and use expo-notifications for display only via `scheduleNotificationAsync` in a `setBackgroundMessageHandler`:

```
import messaging from "@react-native-firebase/messaging";
import * as Notifications from "expo-notifications";

messaging().setBackgroundMessageHandler(async (remoteMessage) => {
  await Notifications.scheduleNotificationAsync({
    content: {
      title: remoteMessage.notification?.title ?? "YTC Alumni",
      body:  remoteMessage.notification?.body ?? "",
      data:  remoteMessage.data ?? {},
      sound: "default",
    },
    trigger: null,
  });
});

// Subscribe topics on YTC sign-in:
await messaging().subscribeToTopic("all_users");
await messaging().subscribeToTopic("new_shiurim");
await messaging().subscribeToTopic("events");
await messaging().subscribeToTopic("announcements");

// Unsubscribe on YTC sign-out / lock.
```

F.4 Trade vs path A

- Pros: no backend bridge; topic subscription is the canonical path used by native YTC apps.
- Cons: native module → custom dev client rebuild; manifest-merge collision must be tested on every EAS build; two notification systems on the device increase the surface for regressions.

End of document.